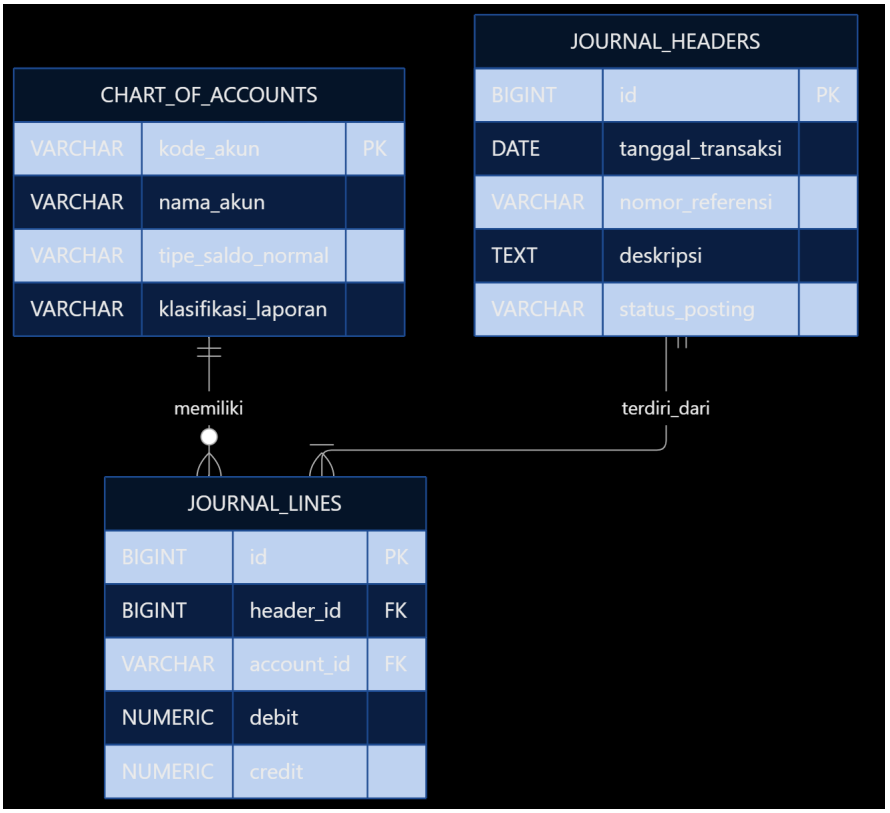
PENGKODEAN DAN PEMOGRAMAN
Naila Qanitah Syahad (12030124130094)
Dosen Pengampu: Dr. Totok Dewayanto, S.E., M.Si., Akt.

Link Github: https://github.com/nailasyahad-oss/UTS-Kode-094

1. Perancangan Skema Relasional (ERD)
   Kardinalitas relasi ERD:
   1.1. journal_headers ke journal_lines adalah 1:N. Satu bukti jurnal punya minimal dua baris, yaitu satu debit dan satu kredit.
   1.2. chart_of_accounts ke journal_lines adalah 1:N. Satu akun bisa dipakai di banyak baris jurnal.
   1.3. journal_lines menjadi tabel penghubung antara jurnal dan akun.



2. DDL SQL
   2.1. Penjelasan bagian penting kode
      2.1.1. PRIMARY KEY
      Setiap tabel memiliki identitas unik. Contohnya, header_id membedakan satu jurnal dari jurnal lainnya, sedangkan line_id membedakan setiap rincian jurnal.
      2.1.2. FOREIGN KEY dan ON DELETE RESTRICT

Pada tabel journal_lines, header_id harus mengacu pada jurnal yang sudah ada dan account_id harus mengacu pada akun yang sudah terdaftar.

Klausa ON DELETE RESTRICT mencegah header atau akun dihapus selama masih direferensikan oleh baris jurnal. Dengan demikian, referensi transaksi historis tetap terjaga.

2.1.3. CHECK (debit >= 0) dan CHECK (credit >= 0)

Kedua constraint tersebut mencegah nilai nominal negatif. Nilai nol tetap diperbolehkan karena satu baris jurnal biasanya hanya mempunyai nominal debit atau nominal kredit.

2.1.4. CHECK (debit > 0 OR credit > 0)

2.1.5. Constraint ini menolak baris jurnal apabila debit dan kredit sama-sama nol. Contoh:

Debit	Kredit	Hasil
100.000	0	Diterima
0	100.000	Diterima
100.000	100.000	Diterima oleh constraint ini
0	0	Ditolak
-100.000	0	Ditolak

2.2. Bagaimana memastikan total debit sama dengan total kredit?

Setiap baris di atas lolos dari constraint nominal, tetapi jurnalnya tidak seimbang. Karena itu, CHECK biasa pada satu baris tidak cukup untuk memastikan bahwa total debit sama dengan total kredit pada seluruh baris dengan header_id yang sama. Solusi yang tepat adalah menambahkan validasi pada proses posting jurnal menggunakan stored procedure atau constraint trigger yang memeriksa:

$$\sum \text{Debit} = \sum \text{Kredit}$$

Jurnal baru boleh berubah menjadi POSTED apabila total debit sama dengan total kredit dan terdapat rincian jurnal yang valid. Implementasi validasi juga harus menangani transaksi bersamaan agar dua proses tidak dapat melewati pemeriksaan secara bersamaan. Bagian ini memperkuat penerapan double-entry bookkeeping yang menjadi inti masalah dalam soal.

3. Analisis audit dan keamanan finansial

3.1. Mengapa model Header-Line lebih andal daripada tabel tunggal?

Model Header-Line memisahkan informasi umum transaksi dari rincian akun yang terlibat dalam transaksi. Model ini lebih tepat untuk sistem

General Ledger karena satu transaksi akuntansi dapat melibatkan dua atau lebih akun. Perbandingannya sebagai berikut:

Aspek	Model Header-Line	Model Tabel Tunggal (Flat Table)
Struktur data	Header dan rincian dipisahkan	Informasi transaksi dan rincian akun dicampur
Transaksi dengan banyak akun	Dapat menyimpan banyak baris melalui satu header_id	Informasi header berulang pada setiap baris
Redundansi	Mengurangi pengulangan tanggal, nomor bukti, dan deskripsi	Berpotensi mengulang informasi transaksi
Konsistensi data	Informasi umum transaksi disimpan pada satu header	Perubahan informasi header berpotensi menimbulkan inkonsistensi
Audit trail	Identitas transaksi dapat ditelusuri melalui header dan rincian	Penelusuran transaksi lebih sulit ketika data umum dan detail bercampur

Sebagai contoh, transaksi pembayaran perlengkapan sebesar Rp100.000 memerlukan dua baris jurnal: debit Perlengkapan dan kredit Kas. Dalam model Header-Line, kedua baris tersebut terhubung ke satu header sehingga keduanya diketahui sebagai bagian dari transaksi yang sama.

Sebaliknya, jika menggunakan satu tabel tunggal, tanggal, nomor bukti, dan deskripsi transaksi harus diulang untuk setiap baris jurnal. Pengulangan tersebut meningkatkan risiko ketidakkonsistenan apabila suatu informasi diperbarui hanya pada sebagian baris.

3.2. Hubungan dengan normalisasi bentuk normal ketiga (3NF)

Normalisasi adalah proses pengorganisasian data untuk mengurangi redundansi dan mencegah anomali ketika data ditambahkan, diperbarui, atau dihapus. Model yang dirancang dapat dijelaskan melalui tiga tahap berikut:

3.2.1. Bentuk normal pertama (1NF)

Setiap kolom menyimpan satu nilai, dan setiap baris dapat diidentifikasi secara unik melalui Primary Key. Rincian jurnal

disimpan sebagai baris terpisah, bukan sebagai daftar akun dalam satu kolom.

3.2.2. Bentuk normal kedua (2NF)

Seluruh atribut non-key harus bergantung pada keseluruhan kunci kandidat, bukan hanya sebagian dari kunci komposit. Dalam rancangan ini, setiap tabel menggunakan Primary Key tunggal sehingga tidak terdapat ketergantungan parsial terhadap Primary Key tersebut.

3.2.3. Bentuk normal ketiga (3NF)

Atribut non-key harus bergantung pada kunci, keseluruhan kunci, dan tidak bergantung secara transitif pada kunci melalui atribut non-key lainnya.

Sebagai contoh, account_name dan normal_balance disimpan di chart_of_accounts, bukan diulang pada setiap baris journal_lines. Sementara itu, transaction_date dan description disimpan di journal_headers, bukan diulang untuk setiap rincian jurnal.

Pemisahan ini membantu memastikan bahwa setiap atribut disimpan pada tabel yang sesuai dengan ketergantungan datanya.

Kesimpulan normalisasi: rancangan Header-Line mendukung penerapan 3NF karena informasi akun, informasi transaksi, dan rincian jurnal ditempatkan pada entitas yang berbeda. Meskipun demikian, pemenuhan 3NF secara formal tetap bergantung pada seluruh ketergantungan fungsional yang berlaku pada skema tersebut.

3.3. Bagaimana model ini mendukung audit trail?

Audit trail adalah kemampuan untuk menelusuri transaksi berdasarkan bukti dan rincian pencatatannya. Model Header-Line mendukung audit trail melalui beberapa mekanisme:

1. reference_no mengidentifikasi nomor bukti transaksi.
2. transaction_date menunjukkan tanggal transaksi.
3. description memberikan keterangan mengenai transaksi.
4. header_id menghubungkan informasi umum transaksi dengan seluruh baris jurnalnya.
5. account_id menunjukkan akun yang digunakan dalam setiap baris.
6. posting_status menunjukkan apakah jurnal masih berstatus DRAFT atau sudah POSTED.

Sebagai ilustrasi, auditor yang memeriksa pembayaran perlengkapan dapat mencari nomor bukti transaksi, melihat tanggal dan deskripsinya,

kemudian menelusuri baris debit Perlengkapan serta kredit Kas melalui header_id.

Namun, perlu dibedakan bahwa struktur Header-Line mendukung penelusuran transaksi, tetapi tidak otomatis menjamin seluruh riwayat perubahan tercatat. Untuk audit trail yang lebih kuat, sistem perlu dilengkapi dengan log perubahan, identitas pengguna, waktu perubahan, dan pengendalian agar jurnal yang telah dibukukan tidak diubah tanpa prosedur yang sah.

SOAL 2

1. Otomatisasi mutasi melalui database trigger

1.1. Memahami masalahnya

Soal memberikan dua tabel:

a. inventory_items: menyimpan data barang, jumlah stok, dan biaya per unit.
b. inventory_transactions: menyimpan transaksi PURCHASE, SALE, atau ADJUSTMENT.

Targetnya adalah setiap kali transaksi baru dimasukkan ke inventory_transactions, database otomatis memperbarui stock_qty pada inventory_items. Contoh sederhana:

Jenis transaksi	Stok awal	Kuantitas	Stok akhir
PURCHASE	10	5	15
SALE	10	3	7
SALE melebihi stok	2	5	Ditolak

Dengan trigger, aplikasi tidak perlu mengubah stok secara terpisah setelah memasukkan transaksi. Database yang menjalankan logika tersebut secara otomatis.

1.2. Penjelasan kode

Bagian Kode	Fungsi
CREATE OR REPLACE FUNCTION	Membuat atau mengganti fungsi trigger.
RETURNS TRIGGER	Menandakan fungsi digunakan oleh trigger.
NEW	Mengakses data transaksi yang baru dimasukkan.
SELECT ... FOR UPDATE	Mengunci baris barang selama transaksi berlangsung.
CASE	Menentukan perubahan stok berdasarkan jenis transaksi.
RAISE EXCEPTION	Menolak transaksi apabila kuantitas tidak valid, barang tidak ditemukan, atau stok menjadi negatif.

UPDATE inventory_items	Memperbarui stok di database.
AFTER INSERT	Menjalankan fungsi setelah baris transaksi dimasukkan.

Catatan penting mengenai nama fungsi: soal meminta fungsi bernama trg_update_stock_and_journal(), tetapi struktur tabel yang diberikan belum mempunyai tabel jurnal akuntansi maupun informasi akun debit-kredit. Karena itu, kode di atas benar-benar memperbarui stok, tetapi belum menghasilkan jurnal akuntansi. Untuk membuat jurnal otomatis, diperlukan tambahan struktur jurnal dan pemetaan akun. Jangan mengklaim bahwa jurnal sudah tercatat jika tabel dan logika pencatatannya belum dibuat.

Kuantitas ADJUSTMENT pada implementasi ini menggunakan tanda positif untuk penambahan dan negatif untuk pengurangan stok. Konvensi ini perlu dijelaskan karena naskah ujian tidak menetapkan bagaimana kuantitas penyesuaian harus direpresentasikan.

2. Penanganan concurrency

2.1. Apa masalahnya?

Misalnya stok barang hanya tersisa 1 unit, lalu dua kasir memproses penjualan masing-masing 1 unit hampir bersamaan. Tanpa penguncian yang benar, kedua kasir berpotensi membaca stok awal yang sama, sehingga total penjualan menjadi 2 unit meskipun stok hanya 1 unit. Kondisi ini disebut overselling.

2.2. Solusi menggunakan SELECT ... FOR UPDATE

SELECT ... FOR UPDATE mengunci baris barang yang sedang diproses. Transaksi lain yang ingin mengunci baris yang sama harus menunggu sampai transaksi pertama selesai.

Jika stok awal hanya 1 unit:

1. Kasir A mendapatkan kunci baris dan memproses penjualan.
2. Kasir B menunggu karena baris barang sedang dikunci.
3. Setelah Kasir A melakukan COMMIT, Kasir B membaca stok terbaru ketika memperoleh kunci.
4. Jika stok sudah 0, trigger menjalankan RAISE EXCEPTION dan transaksi Kasir B dibatalkan.

Kode contoh ini mengasumsikan aplikasi memeriksa hasil pembacaan stok sebelum melakukan INSERT. Trigger tetap menjadi pengendali

utama yang mencegah stok negatif, sehingga validasi tidak hanya bergantung pada aplikasi.

Penting: penguncian harus berlangsung dalam transaksi yang sama dengan proses penjualan. Jika penguncian dilepas sebelum penjualan diproses, perlindungannya tidak memadai.

2.3. Alternatif menggunakan isolasi SERIALIZABLE

Alternatif lainnya adalah menjalankan transaksi pada tingkat isolasi SERIALIZABLE. Pada tingkat isolasi ini, PostgreSQL memastikan transaksi yang berhasil dapat diperlakukan seolah-olah berjalan dalam urutan serial. Jika terjadi konflik, salah satu transaksi dapat gagal dengan serialization error dan harus diulang oleh aplikasi.

Walaupun demikian, SERIALIZABLE bukan pengganti penanganan error. Aplikasi harus siap mengulang transaksi yang gagal. Dalam kasus persediaan sederhana, SELECT ... FOR UPDATE pada baris barang yang tepat merupakan pendekatan langsung untuk mengendalikan akses bersamaan.

2.4. Perbandingan kedua metode

Aspek	SELECT ... FOR UPDATE	SERIALIZABLE
Cara kerja	Mengunci baris tertentu	Mengendalikan konflik berdasarkan isolasi serial
Keunggulan	Efisien untuk mengamankan stok barang tertentu	Memberikan jaminan isolasi transaksi yang lebih ketat
Konsekuensi	Transaksi lain yang membutuhkan baris terkunci harus menunggu	Transaksi dapat gagal dan perlu diulang
Penerapan	Cocok untuk transaksi stok dengan item tertentu	Cocok ketika aturan transaksi lebih kompleks

SOAL 3

1. Kesesuaian Komponen Kas dan Setara Kas
   1.1. Status: Sesuai
   1.2. Verifikasi: Standar akuntansi mendefinisikan kas sebagai saldo kas tunai (cash on hand) dan rekening giro (demand deposits), sedangkan setara kas adalah investasi jangka pendek yang sangat likuid (berjangka waktu 3 bulan atau kurang sejak perolehan) dan bebas dari risiko perubahan nilai yang signifikan. Pemilihan akun Kas Besar/Kecil maupun Rekening Bank pada form voucher kas keluar memenuhi kriteria komponen kas dan setara kas yang digunakan untuk memenuhi komitmen operasional jangka pendek.

2. Pengakuan Arus Kas Keluar Aktivitas Operasi
   2.1. Status: Sesuai
   2.2. Verifikasi: Sesuai ketentuan PSAK 2 / PSAK 207, pengeluaran kas untuk pembayaran kepada pemasok, gaji karyawan, serta beban operasional harian diklasifikasikan sebagai pengeluaran kas dari aktivitas operasi. Transaksi melalui voucher kas keluar mencerminkan penurunan manfaat ekonomik masa depan berupa deplesi kas/setara kas yang diakui sebagai beban atau pelunasan liabilitas.

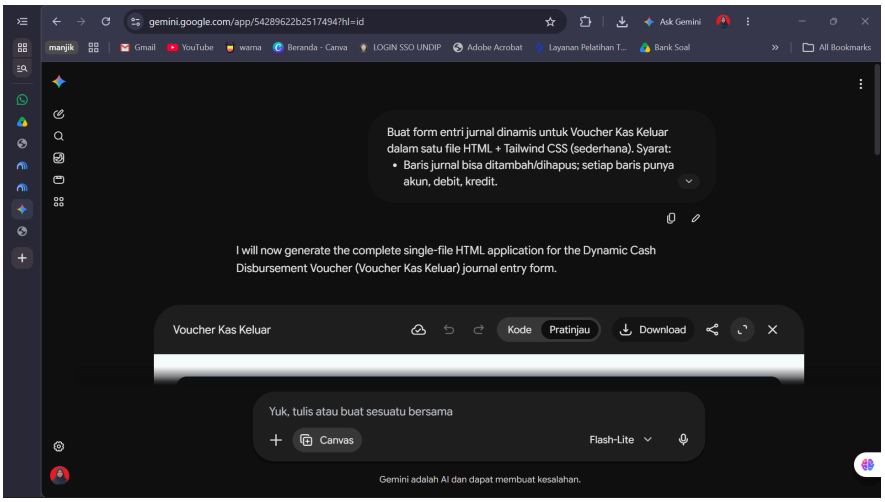
3. Validasi Keseimbangan Pencatatan (Double-Entry System)
   3.1. Status: Sesuai
   3.2. Verifikasi: Pengecekan keseimbangan nominal transaksi (totalDebit === totalKredit) sebelum form diposting mendukung keandalan pencatatan akuntansi dan keterandalan penyusunan laporan keuangan.

4. Catatan & Rekomendasi Pengendalian Internal (Asersi Audit)
   4.1. Validasi Posisi Normal Kas Keluar: Secara logika akuntansi, transaksi pengeluaran kas wajib menempatkan akun Kas/Bank di posisi Kredit (berkurang) dan akun Beban/Utang/Aset di posisi Debit. Sistem perlu memvalidasi posisi akun, bukan hanya keseimbangan matematika nominal.
   4.2. Penanganan Kas yang Dibatasi (Restricted Cash): Standar akuntansi mengatur bahwa saldo kas atau bank yang penggunaannya dibatasi oleh aturan hukum atau kontrak (restricted cash) tidak boleh disajikan atau dikelola sebagai kas bebas tanpa pengungkapan khusus.
   4.3. Verifikasi Bukti Pendukung (Asersi Audit): Untuk memenuhi asersi eksistensi dan kelengkapan, pengeluaran kas melalui voucher harus

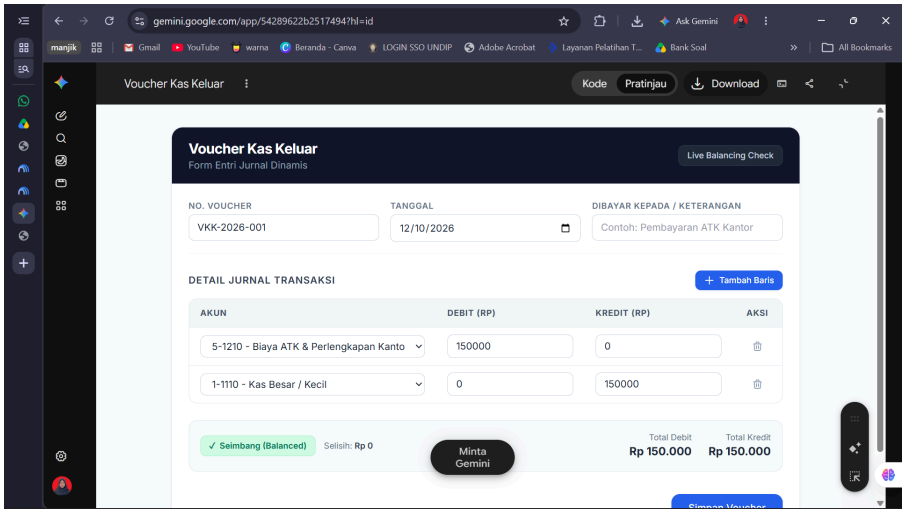
diverifikasi dengan dokumen pendukung yang sah (seperti kuitansi, nota, atau bukti transfer) serta otorisasi pihak berwenang.

Dokumentasi keseluruhan

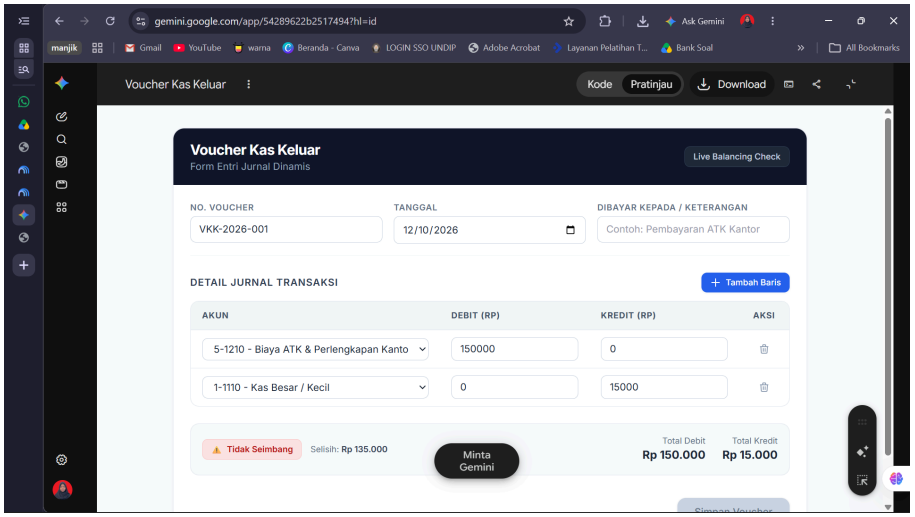
1. Prompt Gemini pembuatan HTML



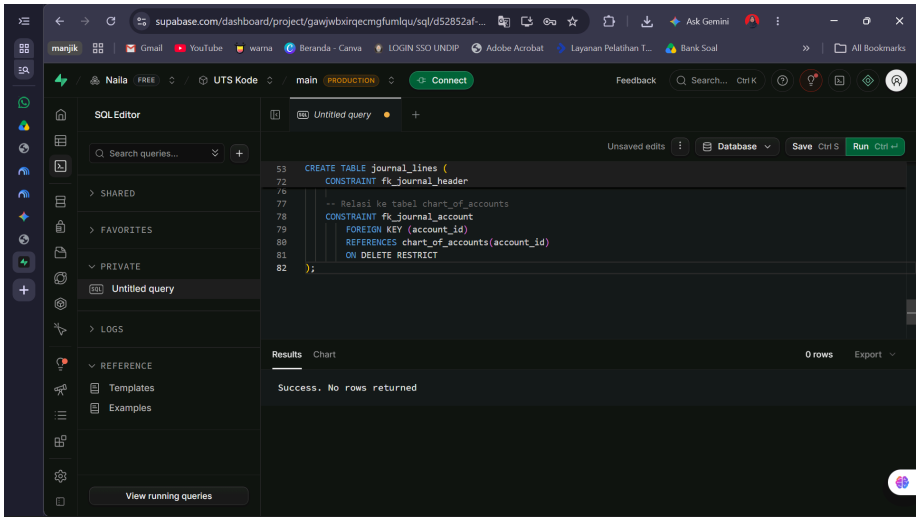
2. Tampilan balance



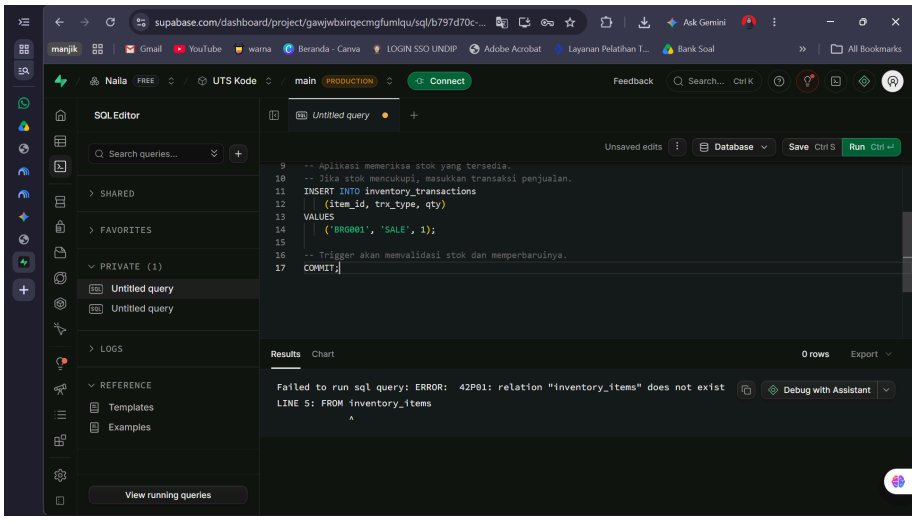
3. Tampilan Tidak Balance



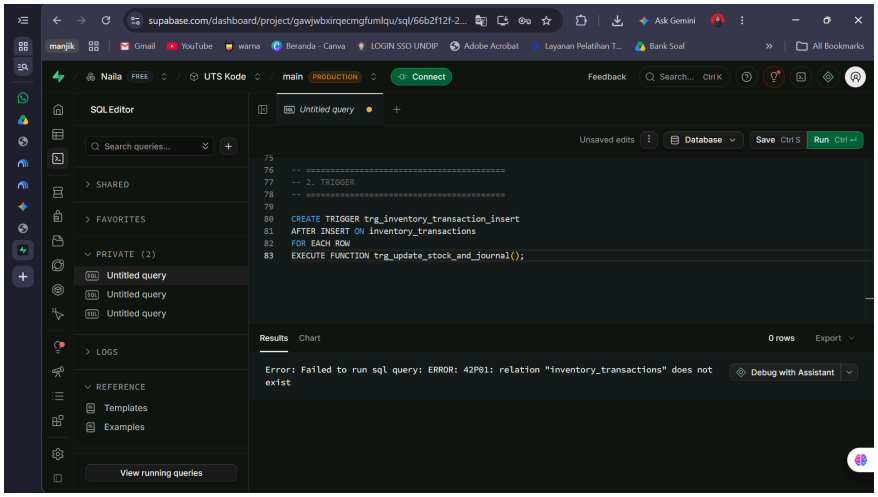
4. Supabase DDL PostgreSQL



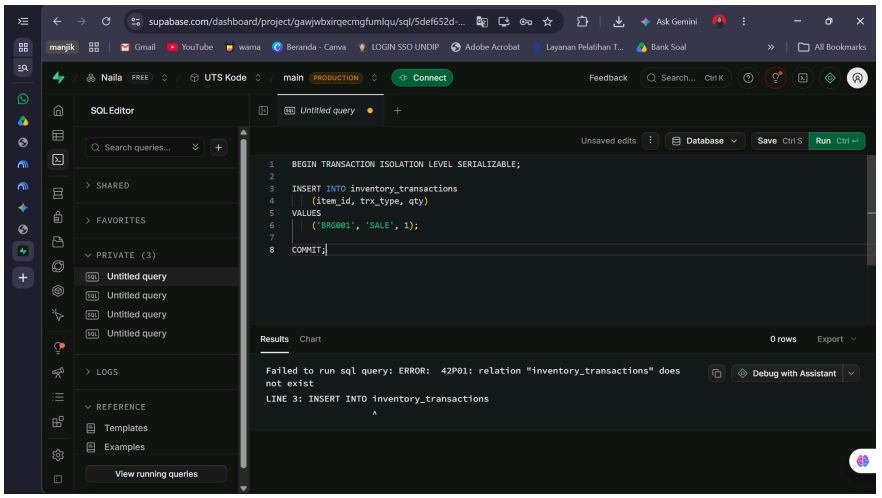
5. SQL Concurrency SELECT ... FOR UPDATE



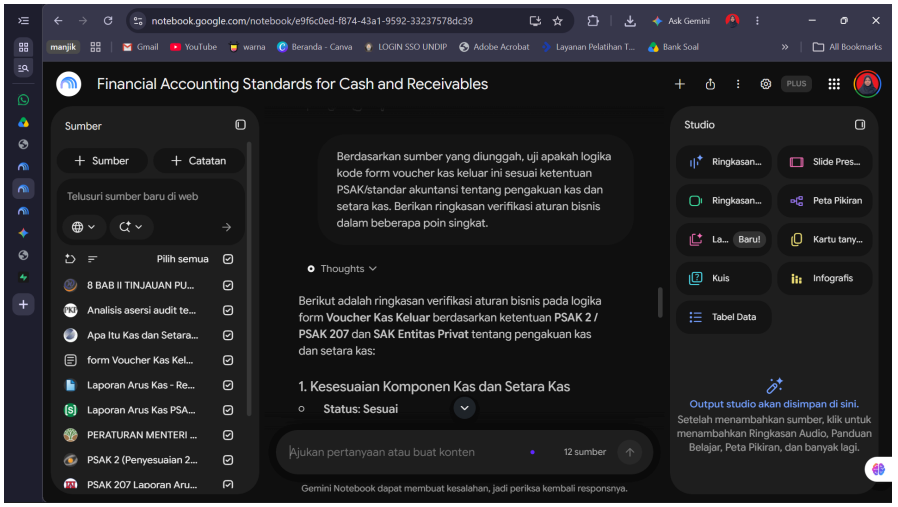
6. Trigger Function



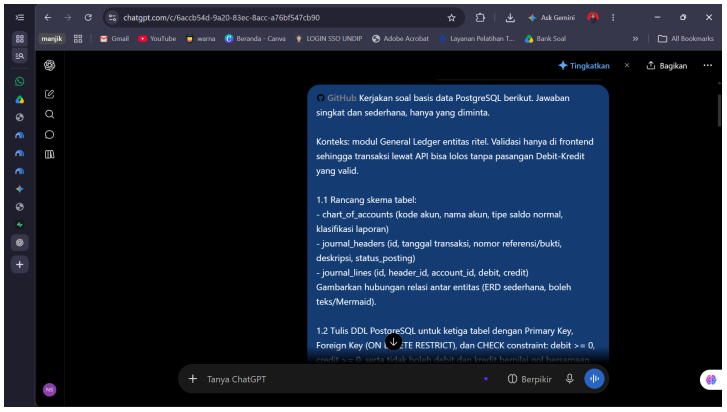
7. Serialize

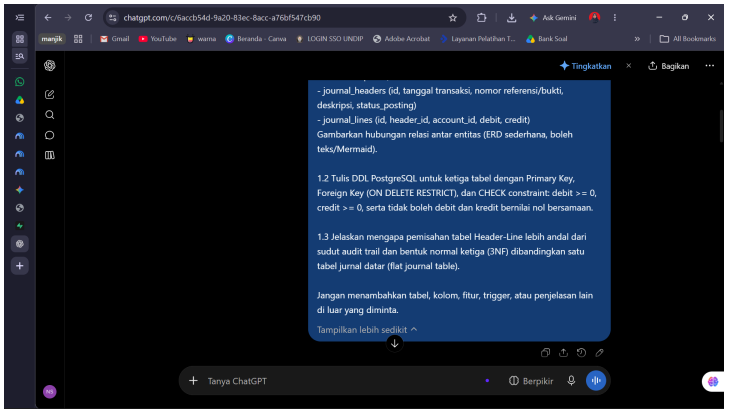


8. Prompt Notebooklm

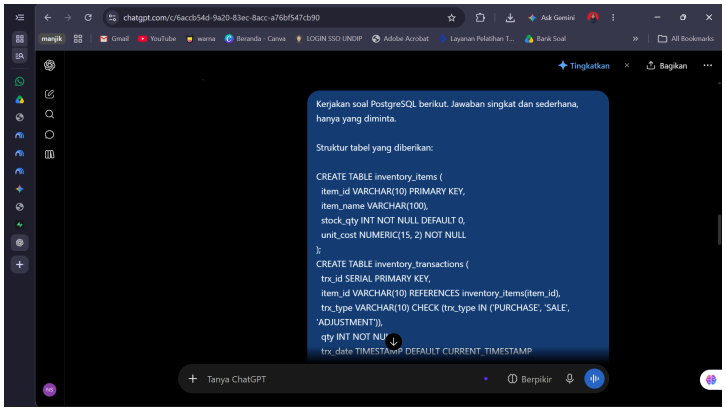


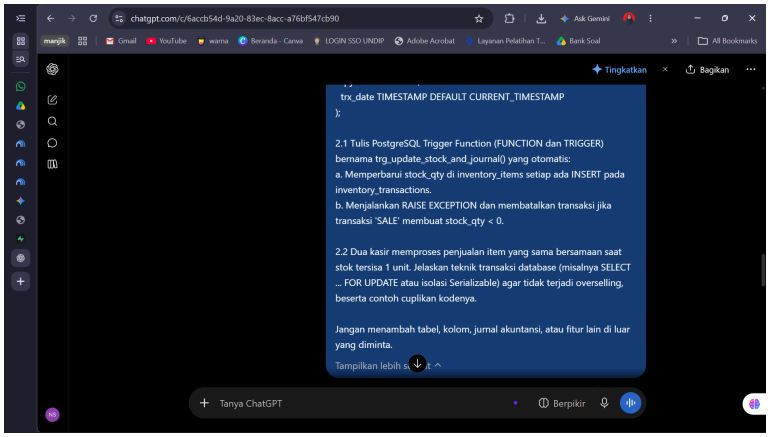
9. Prompt Soal 1





10. Prompt Soal 2





11. Prompt Soal 3

